

Simplified EMR Project Outline

Plain Java console application for an Object-Oriented Programming semester project

OOP-FOCUSED	CONSOLE-BASED	ARRAYLIST STORAGE	FILE PERSISTENCE
--------------------	----------------------	------------------------------	-------------------------

Scope confirmed with professor

ArrayList is acceptable in place of a traditional array. The application may remain console-based. Recursion is not required.

Prepared for Hasan | September 2026

1. Project purpose and final scope

Build a small but complete Electronic Medical Record system that lets a user manage patients, doctors, appointments, and basic medical records through a console menu. The application demonstrates the course's central OOP concepts without introducing framework-level architecture.

Scope decision

This is the class version, not the final portfolio version. Completion, correctness, and clear OOP design take priority over repositories, generics, Spring Boot, databases, or a graphical interface.

Required features

- Add and view patients.
- Add and view doctors.
- Find a patient or doctor by ID.
- Schedule appointments that connect a patient and doctor.
- Prevent a doctor from having two appointments at the same date and time.
- View a patient's appointments and a doctor's schedule.
- Cancel or complete an appointment.
- Add a medical record to a patient and view the patient's medical history.
- Save data to files and reload it during the next run.
- Validate input and handle ordinary errors without crashing.

Deliberately excluded

- Spring Boot, REST APIs, repositories, generic repository interfaces, databases, JDBC, JPA, and React.
- GUI programming, event listeners, recursion, authentication, billing, and insurance claims.
- Multiple medical-record subtypes, advanced reports, recurring appointments, and multi-user access.
- Real patient information or claims that the project is suitable for clinical use.

2. Course-requirement alignment

The confirmed project scope meets the applicable syllabus objectives through natural application behavior.

Course concept	Project evidence
Variables, types, assignments, and methods	Fields, local variables, IDs, dates, statuses, constructors, and method calls throughout the application.
Conditionals and loops	Menu decisions, validation rules, searches, conflict checks, menu loops, and ArrayList traversal.
Collections and iterators	ArrayList collections for patients, doctors, appointments, and medical records; enhanced loops or iterators for traversal.
Objects and classes	Concrete Patient, Doctor, Appointment, and MedicalRecord objects created from class definitions.
Exceptions	Invalid input, unknown IDs, scheduling conflicts, and file failures are handled predictably.
Packages	Classes are grouped into model, application, UI, storage, and exception packages.
Encapsulation	Private fields, constructor initialization, getters, and controlled setters or state-change methods.
Abstraction	Abstract Person class; EMRSystem and FileManager expose simple operations while hiding collections and file details.
Inheritance	Patient and Doctor extend Person.
Polymorphism	Patient and Doctor override the same Person display operation; calls through Person references produce type-specific results.
Strong specifications / ADTs	Public methods have defined inputs, outcomes, validation rules, and exceptions; collections remain hidden inside owning classes.

Professor-confirmed substitutions

ArrayList satisfies the data-structure expectation; a console application satisfies the project interface; recursion does not need to appear.

3. Package and class inventory

A modest package structure keeps responsibilities clear without adding enterprise-style layers.

Package	Class	Primary responsibility
model	Person	Abstract shared identity and polymorphic display contract.
model	Patient	Patient details and owned medical-record collection.
model	Doctor	Doctor details such as specialty and license number.
model	Appointment	Patient-doctor scheduling relationship and controlled status changes.
model	AppointmentStatus	Scheduled, completed, and cancelled values.
model	MedicalRecord	One dated diagnosis/notes entry authored by a doctor.
application	EMRSystem	Owns ArrayLists and coordinates all system operations and business rules.
ui	MenuManager	Displays menus, reads input, calls EMRSystem, and presents results.
storage	FileManager	Saves and reloads the system's data.
exception	RecordNotFoundException	Reports an unknown entity ID.
exception	AppointmentConflictException	Reports a doctor's occupied date/time.
exception	InvalidDataException	Reports invalid domain or stored data.
root	EMRApplication	Creates objects, loads data, starts the menu, and saves before exit.

Dependency direction

EMRApplication constructs the system. MenuManager interacts with EMRSystem. EMRSystem manages model objects. FileManager reads and writes application data. Model classes do not read input or access files.

Design boundary

MenuManager should never search ArrayLists directly, check scheduling conflicts, or write files. It gathers input and delegates those tasks.

4. Model classes: people

Person - abstract class

Purpose: define state and behavior shared by every person in the system. A plain Person is never created.

Fields	Method responsibilities
ID: String, immutable First name: String Last name: String Date of birth: LocalDate	Initialize and validate common fields. Return ID, names, date of birth, and full name. Allow controlled name/date updates if required. Define an abstract details-display operation. Define identity equality using ID.

Patient - extends Person

Additional fields	Method responsibilities
Phone number: String Emergency contact: String Medical history: ArrayList of MedicalRecord	Initialize patient-specific data. Return and update contact information. Add a valid medical record. Return a safe view or copy of medical history. Override the Person details operation with patient-specific information.

Doctor - extends Person

Additional fields	Method responsibilities
Specialty: String License number: String, immutable	Initialize and validate doctor data. Return or update specialty. Return the license number. Override the Person details operation with doctor-specific information.
Explicit polymorphism EMRSystem may process a Patient or Doctor through a Person reference and call the same details operation. Dynamic dispatch selects the correct overridden behavior.	

5. Model classes: clinical activity

Appointment

Fields	Method responsibilities
Appointment ID: String, immutable Patient: Patient reference Doctor: Doctor reference Date/time: LocalDateTime Reason: String Status: AppointmentStatus	Create a scheduled appointment. Return all appointment information. Reschedule while still scheduled. Cancel a scheduled appointment. Complete a scheduled appointment. Reject invalid status changes. Report whether it occupies a scheduling slot.

AppointmentStatus - enum

- SCHEDULED - active appointment that blocks the doctor's time slot.
- COMPLETED - historical appointment; no further state changes.
- CANCELLED - historical appointment that no longer blocks the time slot.

MedicalRecord

Fields	Method responsibilities
Record ID: String, immutable Entry date: LocalDate Author: Doctor reference Diagnosis: String Notes: String	Create and validate a record entry. Return all stored information. Produce a formatted record summary for console display. Remain associated with the Patient that owns it.

Relationship decisions

- Patient is a Person: inheritance.
- Doctor is a Person: inheritance.
- Patient has medical records: composition.
- Appointment connects an independent Patient and Doctor: association.
- MedicalRecord refers to its authoring Doctor: association.

6. EMRSystem: application data and rules

Purpose: act as the application's main abstract data type. It hides the internal ArrayLists and exposes meaningful EMR operations.

Fields

- ArrayList of Patient objects
- ArrayList of Doctor objects
- ArrayList of Appointment objects
- Next patient, doctor, appointment, and record ID counters, or equivalent ID-generation state

Patient operations

Operation	Responsibility
Add patient	Validate uniqueness, create or accept the patient, and add it to the collection.
Find patient by ID	Return the matching patient or raise RecordNotFoundException.
List patients	Return a safe list for display.
Add medical record	Find patient and doctor, create the record, and add it to the patient's history.

Doctor operations

Operation	Responsibility
Add doctor	Validate ID and license uniqueness and add the doctor.
Find doctor by ID	Return the matching doctor or raise RecordNotFoundException.
List doctors	Return a safe list for display.

Appointment operations

Operation	Responsibility
Schedule appointment	Resolve patient and doctor, validate the date/time, check conflict, and add appointment.
Find appointment by ID	Return the appointment or raise RecordNotFoundException.
Find patient appointments	Return appointments associated with one patient.
Find doctor appointments	Return a chronological doctor schedule.
Check scheduling conflict	Detect another scheduled appointment for the doctor at the exact same date/time.
Cancel or complete appointment	Delegate the valid state change to Appointment.

7. Console, storage, and startup

MenuManager

Contains one shared Scanner and references the EMRSystem. It runs the main loop and patient, doctor, appointment, and medical-record submenus.

- Read required text, numeric choices, dates, and date/time values with retry loops.
- Add and display patients and doctors.
- Schedule, view, cancel, and complete appointments.
- Add and display medical records.
- Catch expected project exceptions and print understandable messages.
- Return to previous menus and exit cleanly.

FileManager

Receives or returns the application's data without containing business rules.

- Save patients, doctors, appointments, and records to documented text files.
- Load stored data when the program starts.
- Reconstruct relationships between loaded patients, doctors, appointments, and records.
- Handle a missing file as an empty first run.
- Translate malformed data and I/O failures into controlled errors.
- Use fictitious data only.

EMRApplication

- Create EMRSystem and FileManager.
- Load saved data into the system.
- Create MenuManager using the loaded system.
- Start the console loop.
- Save data before a normal exit.
- Display a clear startup message if stored data cannot be loaded safely.

Persistence timing

Implement file persistence only after the complete application works in memory. This separates OOP debugging from file-format debugging and reduces project risk.

8. Exceptions and method specifications

Focused exception set

Exception	Use
RecordNotFoundException	A requested patient, doctor, or appointment ID does not exist.
AppointmentConflictException	The selected doctor already has a scheduled appointment at the requested date/time.
InvalidDataException	Required fields, date values, status changes, or stored records are invalid.
Standard input/file exceptions	Scanner parsing and file operations are caught at suitable boundaries.

Strong method specifications

Before implementing an important public method, define four things in its Javadoc or design notes:

- ☐ Inputs and their valid ranges or required conditions.
- ☐ Successful result or change to application state.
- ☐ Conditions that leave state unchanged.
- ☐ Exceptions or error outcomes the caller must handle.

Example specification in plain language

Scheduling an appointment requires an existing patient, an existing doctor, a future date/time, a nonblank reason, and an unoccupied doctor time slot. Success adds exactly one scheduled appointment. Any failed validation adds nothing and produces the relevant error.

Encapsulation rules

- All model fields remain private.
- IDs and license numbers do not change after construction.
- Getters do not expose mutable internal ArrayLists directly.
- Appointment status changes occur through Appointment methods, not through a public status setter.
- Medical records are added through Patient or EMRSystem behavior.
- The menu never changes model fields directly.

9. Small-chunk construction plan

Finish and test each chunk before starting the next. Do not add optional features while a required chunk remains unstable.

Chunk	Build	Completion gate
1	Project shell: Create packages, EMRApplication, and a temporary startup/exit flow.	The project compiles, runs, and has a short README scope statement.
2	Person hierarchy: Build Person, Patient, and Doctor; test construction, validation, inheritance, and overridden details.	Valid people work and invalid required values are rejected.
3	Appointments: Build AppointmentStatus and Appointment; test scheduling state and allowed transitions.	Appointment behavior works with manually created people.
4	Medical records: Build MedicalRecord and Patient medical-history behavior.	A patient can own and display several records.
5	EMRSystem people: Add patient/doctor collections, IDs, add/find/list operations, and exceptions.	All people operations work without a menu.
6	EMRSystem scheduling: Add appointment collection, conflict checks, schedules, histories, cancel, and complete operations.	All appointment rules work without a menu.
7	Console interface: Build menus and reusable validated input loops for every required workflow.	The full project works during one program run without ordinary input crashes.
8	File persistence: Save and load every entity and reconstruct object relationships.	Complete system state survives a restart.
9	Testing and submission: Run edge cases, clean code, finish README, and prepare a short demonstration.	The acceptance scenario passes from a clean start.

Deadline protection

If persistence is not complete two weeks before the deadline, freeze features. Finish loading/saving, testing, documentation, and the demonstration instead of expanding scope.

10. Acceptance checklist

Required user flow

- ☐ Start with no data files and treat the system as empty.
- ☐ Create at least two patients and two doctors.
- ☐ Find each person by ID.
- ☐ Schedule an appointment connecting an existing patient and doctor.
- ☐ Reject a second appointment for that doctor at the same date and time.
- ☐ Display the patient's appointments and the doctor's schedule.
- ☐ Cancel or complete an appointment using a valid state transition.
- ☐ Add a medical record to a patient and display the patient's history.
- ☐ Exit, restart, and verify that data and relationships reload correctly.

OOP evidence

- ☐ Person is abstract and cannot be instantiated directly.
- ☐ Patient and Doctor inherit common fields and behavior.
- ☐ Patient and Doctor override the same Person operation.
- ☐ At least one call through a Person reference demonstrates runtime polymorphism.
- ☐ Fields are private and important state changes use controlled methods.
- ☐ Patient owns an ArrayList of medical records.
- ☐ EMRSystem hides its ArrayLists and exposes clear operations.
- ☐ Expected errors use exceptions and are handled by the console boundary.
- ☐ Classes are placed in coherent packages.
- ☐ Important public operations have clear specifications.

Submission quality

- ☐ No debugging output or temporary test code remains.
- ☐ README explains setup, features, OOP concepts, file locations, and limitations.
- ☐ Only fictitious demonstration data is used.
- ☐ A manual test checklist records expected and actual results.
- ☐ A five-to-eight-minute demo can be completed from a clean start.

Final success standard

A small application that is complete, reliable, and clearly explained is a stronger class project than a sophisticated architecture that is only partially implemented.

11. Post-class expansion path

Preserve this version as the completed class project. Expand it later through controlled refactoring rather than trying to build the portfolio system now.

Class version	Future portfolio version
EMRSystem owns ArrayLists and rules	Split responsibilities into service and repository layers.
Direct object references	Use IDs, database keys, and JPA relationships.
FileManager	Replace with repository interfaces and PostgreSQL persistence.
Single MedicalRecord class	Introduce an abstract entry hierarchy for diagnoses, prescriptions, and lab results.
Console MenuManager	Replace with Spring Boot REST controllers and a React/TypeScript client.
Manual construction in EMRApplication	Use Spring dependency injection.
Focused exceptions	Map domain exceptions to HTTP responses.

Recommended stopping point for class

- All required console workflows pass.
- Data survives restart.
- Inheritance and polymorphism are visible and explainable.
- The project handles errors cleanly.
- The README and demo are ready.

Only after submission

- Create UML for the refactored portfolio architecture.
- Introduce services and repositories one responsibility at a time.
- Move persistence to PostgreSQL through Spring Boot and JPA.
- Build the REST API, then connect the React/TypeScript frontend.

Scope confirmed from the professor's guidance: ArrayList is acceptable; console interaction is sufficient; recursion is not required.